From Attack Behavior to Defense Failure: A Stage-Aware Framework for Evaluating APT Defense Capability

Anonymous Author(s)

Abstract

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; • **Software and its engineering** → *Software creation and management*.

Keywords

APT defense evaluation, control-point attribution, stage-aware scoring, security benchmark

ACM Reference Format:

Anonymous Author(s). 2026. From Attack Behavior to Defense Failure: A Stage-Aware Framework for Evaluating APT Defense Capability. In *Proceedings of THE ACM WEB CONFERENCE 2026 (Conference acronym 'WWW).* ACM, New York, NY, USA, 9 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Advanced Persistent Threats (APTs) have become a major threat to national, enterprise, and critical-infrastructure cybersecurity. According to M-Trends 2026, the global median dwell time rose to 14 days in 2025, while the median dwell time for cyber-espionage intrusions reached 122 days[4]. Verizon DBIR 2025 reports that credential abuse and vulnerability exploitation remain major initial attack vectors, and that the share of breaches involving third parties doubled to 30%[16]. IBM's 2024 Cost of a Data Breach report further shows that the global average cost of a data breach reached $4.88 million, and that about 70% of affected organizations experienced significant or moderate business disruption[5].

In response, organizations deploy layered defense stacks, including Endpoint Detection and Response (EDR), Next-Generation Firewalls (NGFW), Security Information and Event Management (SIEM), Privileged Access Management (PAM), and email security gateways. However, deployment does not imply effectiveness. A defense stack with more products is not necessarily more capable, and a system that fails to stop an entire APT campaign is not necessarily useless. What matters is where the defense works, where it fails, and whether those failures can be explained in terms that support repair.

APT campaigns are difficult to evaluate because they are multi-stage, long-lived, and operationally diverse. An attacker may enter through phishing, a public-facing vulnerability, or credential abuse, then establish command and control, collect credentials, move laterally, evade detection, and eventually exfiltrate data or cause impact. For evaluation, this means that a single pass/fail result is too coarse. Blocking initial access is not equivalent to detecting exfiltration after compromise. Similarly, an attack stage with several alternative paths should not be scored in the same way as a stage where all required actions must succeed. A useful evaluation should also explain the first defensive control that failed, since the same missed action may point to weak identity controls, insufficient isolation, missing endpoint hardening, or poor observability.

Existing evaluation and scoring approaches address parts of this problem, but they leave the connection between attack behavior and defense failure incomplete. MITRE ATT&CK Evaluations provide detailed step-level observations of product behavior in adversary-emulation scenarios[11]. These results are valuable for understanding whether a product observed, detected, or protected against specific attack steps. They do not, however, directly translate those observations into stage-aware defense scores, nor do they explain which victim-side control point was the first to fail when a step is missed. Other work evaluates system security using vulnerability severity, attack difficulty, risk values, or attack-graph-based reachability[1, 8, 18]. Such methods are useful for risk analysis, but their main concern is usually which vulnerability or attack path is risky, rather than how a deployed defense stack performs against action-level APT behaviors.

The missing piece is an evaluation layer that views each attack action from the defender's side before assigning a score. In this paper, each action is first mapped to the victim-side control point that failed first and is most worth repairing. This choice is motivated by a simple observation: the same ATT&CK technique may require different fixes in different contexts. Credential use, for example, may originate from brute force, credential dumping, hard-coded secrets, or phishing. These cases may share an attack label, but they do not point to the same defensive repair.

Building such a scoring framework raises three practical challenges. The attribution schema must be stable enough to handle heterogeneous APT actions. The attack-stage dimension must be represented without forcing control-point domains into a fixed kill-chain order. Action-level observations also need to be aggregated according to the logic of each stage rather than by a simple average.

This paper proposes a stage-aware APT defense capability scoring framework based on control-point attribution. The framework contains three layers. The first layer classifies each attack action by the victim-side first-failed control point, using 9 top-level domains and 94 leaf-level control points. The second layer turns the attributed actions into scoring-ready benchmark records by adding a canonical ATT&CK phase, a defense opportunity map, and a defense-technology crosswalk that maps control points to defense product categories. The third layer aligns tested defense results with benchmark actions, assigns action-level hit values for `PREVENTED`, `BLOCKED`, `DETECTED`, and `MISSED` outcomes, and aggregates

Conference acronym 'WWW, Dubai, United Arab Emirates

ACM ISBN 978-1-4503-XXXX-X/18/06
https://doi.org/XXXXXXX.XXXXXXX

them into stage-level, playbook-level, phase-wise, control-domain, and product-category scores. Stage logic is modeled explicitly: OR stages use the weakest covered alternative, AND stages use the strongest interruption point, and Cluster stages use weighted averaging.

The main contributions of this paper are as follows:

- We propose a control-point attribution schema for APT defense evaluation, shifting the evaluation unit from "what the attacker did" to "which victim-side control failed first." The schema covers 9 top-level domains and 94 leaf-level control points, enabling action-level attribution of defensive failures.
- We design a benchmark extension architecture that connects control-point attribution with stage-aware scoring. It includes canonical phase mapping, a defense opportunity map, and a defense-technology crosswalk, allowing defense results to be analyzed by attack phase, control domain, and product category.
- We develop a stage-aware scoring model that separates action-level hit values from stage importance and formalizes OR, AND, and Cluster stage semantics. This avoids treating all APT steps as equally valuable and supports aggregation from actions to playbooks.
- We evaluate the framework on 53 APT playbooks containing 1,758 attack actions. The results validate the mathematical behavior of the scoring engine and show that no single product category can independently cover the full APT attack surface, highlighting the need for coordinated defense-in-depth.

2 Methodology

2.1 Overview

This section describes how the proposed method turns APT playbooks and defense observations into scores that can be traced back to concrete actions, stages, and failed control points. The goal is not only to record whether an attack step was detected or blocked. The method also asks why the step should have been stopped: which victim-side control point first failed, which defense categories were expected to cover it, and how the evaluated defense actually responded.

Fig. 1 shows the workflow. It contains three parts: benchmark construction, defense scoring, and score reporting. Benchmark construction starts from raw APT playbook actions. Each action is annotated with a control-point attribution, which identifies the victim-side control point that first failed and is most relevant for repair. The attributed actions are then enriched with canonical attack phases, control-point–to–defense mappings, expected defense product categories, confidence labels, and stage aggregation semantics.

The result is a defense opportunity map. Each record in this map represents one attack action in a form that the scoring engine can use. During defense scoring, the evaluated system provides action-level observation results. These observations are matched with the corresponding opportunity records and converted into four hit levels: `PREVENTED`, `BLOCKED`, `DETECTED`, and `MISSED`. The scoring engine then aggregates these action-level hit values into stage-level and playbook-level scores, using the predefined stage semantics and canonical phase weights.

The final output is not limited to one overall score. The framework also reports scores by attack phase, control-point domain, and product category, together with an action-level attribution report. In this way, a score loss can be traced from the overall score to a playbook, a stage, an action, and finally the failed control point that caused the loss.

2.2 Defensive Weakness Attribution

The first step is to view each attack action from the defender's side. For every in-scope action, we identify the victim-side control point that failed first and is most relevant for repair. A control point may be a defensive mechanism, a configuration, a policy, or an operational process that can be evaluated and remediated independently. A successful attack action therefore indicates that some victim-side control was absent, misconfigured, bypassed, or insufficiently enforced.

This scope excludes actions that do not correspond to a directly repairable victim-side failure. Pre-compromise reconnaissance, attacker-side infrastructure preparation, and similar external prerequisite activities are retained for playbook completeness, but they are not included in primary weakness statistics or attribution-based scoring unless a concrete victim-side control failure can be identified.

This attribution step is needed because an ATT&CK label tells us what the attacker did, but not which defense failed. The same ATT&CK technique may be enabled by different weaknesses in different environments. For example, a credential-related action may result from weak password policy, missing multi-factor authentication, exposed remote login services, insufficient credential storage protection, or successful phishing. These cases may share similar attack labels, but they imply different repair actions.

We therefore interpret each action by asking a repair-oriented question: if only one control point could be fixed, which one would most directly prevent or disrupt this action? The answer is organized through a structured control-point schema. The schema is not an attack sequence. It separates defensive failures along four design dimensions: security function, architectural location, defensive lifecycle phase, and trust source. These dimensions are not intended to form a strictly orthogonal taxonomy. They are used as practical design constraints for separating control points that lead to different repair actions.

Security function follows the long-standing view that protection mechanisms should be separated by their security roles, such as authentication, authorization, privilege separation, execution control, and data protection [15]. Architectural location captures where the relevant trust boundary or enforcement point sits, which is consistent with zero-trust architecture's shift from static network perimeters toward users, assets, resources, and policy enforcement around them [14]. Defensive lifecycle phase distinguishes controls that prevent an action from succeeding from controls that detect, respond to, or recover from successful actions, aligning with the preventive–reactive distinction in cyber defense modeling [19]. Trust source captures whether the failed control is tied to externally supplied inputs, such as phishing messages, malicious documents, third-party collaboration, or supply-chain artifacts; this is

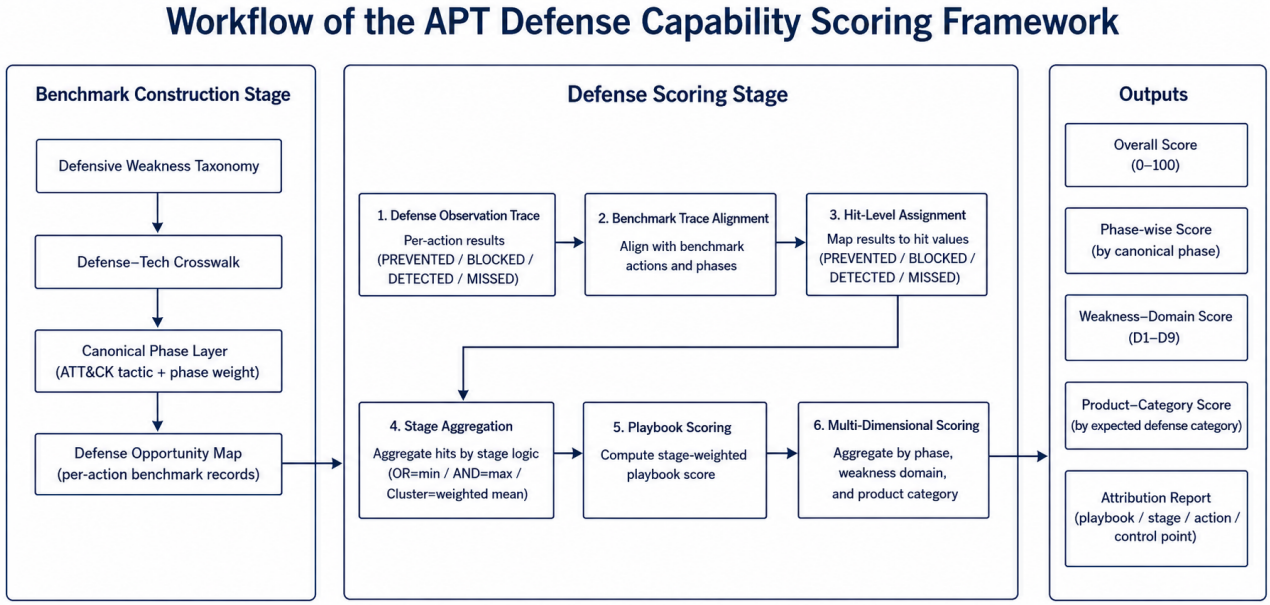

Figure 1: Workflow of the APT defense capability scoring framework.

Table 1: Top-level control-point domains.

Domain	Scope
D1	Identity, credential, and trust-root failures
D2	Authorization, isolation, and boundary-control failures
D3	Execution, host, and runtime-protection failures
D4	External exposure and remote-entry failures
D5	Communication and protocol-design failures
D6	Data protection, cryptographic-material, and recovery failures
D7	Observability, detection, and response failures
D8	External trust-input failures
D9	Availability and resilience failures

related to attack-surface models that treat entry points, channels, and untrusted data items as key resources through which attackers interact with a system [7].

Together, these dimensions help explain why the nine domains are not arranged as a timeline. They describe different ways in which a victim-side control can fail, while the separate canonical phase layer captures when the action occurs in the attack chain.

The schema contains nine top-level control-point domains and 94 fourth-level leaves. The domains provide stable categories for scoring and reporting, while the leaves provide the operational granularity needed for remediation. In other words, the domains explain where score loss is concentrated, and the leaves explain what should be repaired.

Each in-scope action is assigned exactly one primary weakness leaf. This single-label rule keeps later scoring traceable. If one action had multiple primary attributions, the same failure could be counted several times, and score loss could not be linked to a clear remediation target. To keep attribution consistent, the schema uses explicit boundary rules and the repair-oriented decision principle described above.

Some actions still involve more than one relevant weakness. Secondary weakness tags record such supporting conditions, including dual-control dependencies and entry–root-cause splits. These tags help explain attack chains, but they are not counted as primary attribution results and do not replace the primary leaf used for scoring.

After this step, raw attack actions have been converted into repair-oriented control-point failures. This attributed action set is the basis for canonical phase assignment, defense opportunity mapping, and stage-aware scoring.

2.3 Benchmark Construction

Control-point attribution explains which defense failed, but it is not enough for scoring by itself. The scoring engine also needs to know when the action occurs in the attack chain, which defense product categories are expected to address it, and how the action should be aggregated with other actions in the same stage. Benchmark construction adds this information without changing the primary attribution result.

The process has three parts. Attributed actions are first assigned canonical attack phases using ATT&CK. Their control-point leaves are then linked to expected defense product categories. Finally, the enriched records are written into the defense opportunity map, which serves as the direct input to defense scoring.

2.3.1 Canonical Phase Assignment. The attribution domains describe where the victim-side defense failed, not when the action occurs

in the attack chain. Scoring still needs this time-ordering information because blocking an early action and detecting a late action should not have the same defensive value. We therefore add a separate canonical phase layer.

For each action, the framework assigns a canonical phase based on its ATT&CK technique. We use ATT&CK Enterprise tactics as the standard phase vocabulary. Some ATT&CK techniques can be associated with multiple tactics. We resolve this ambiguity using a predefined technique-to-phase mapping. Each technique is mapped to a single canonical phase before scoring, and this mapping remains fixed during defense scoring.

Stage-level phases are then derived from the action-level phases, for example by majority voting among the actions in the same playbook stage. This phase assignment is an additional benchmark field; it does not change the control-point attribution described in the previous subsection. The canonical phase field is later used by the scoring model to apply phase weights. The main intuition is that earlier interruption has higher defensive value, while late-stage detection provides less benefit because the attacker has already progressed through much of the playbook.

2.3.2 Defense-Category Mapping. The primary weakness leaf identifies the failed control point. A scoring benchmark also needs to know which type of defense product should be expected to address that failure. For this purpose, we build a mapping from control-point leaves to defense product categories.

Each fourth-level leaf is mapped to two sets of defense categories. The first set is the primary defense category set. It contains product types that should directly prevent or block the corresponding weakness if deployed and configured correctly. The second set is the compensating defense category set. It contains product types that may detect, mitigate, or partially compensate for the weakness, but are not the most direct repair target.

For example, a brute-force action attributed to weak password and anti-brute-force policy is primarily related to identity and access management, while privileged access management or log monitoring may provide compensating support. Similarly, a C2 communication action attributed to missing egress proxy enforcement may be primarily related to network access control or next-generation firewall capabilities, while SIEM or IDS may provide secondary visibility.

This mapping allows later scoring to evaluate each product category within its expected coverage set. A product is therefore not judged against all benchmark actions equally, but against the actions that its category is expected to cover.

2.3.3 Defense Opportunity Map. The final output of benchmark construction is the defense opportunity map. Each record in the map corresponds to one attack action and combines the information needed for scoring: the playbook identifier, stage number, action description, ATT&CK technique, primary control-point leaf, top-level domain, canonical phase, expected defense categories, confidence label, and stage aggregation semantics.

The opportunity map keeps the primary attribution unique, but it can also record additional defense opportunities when they are available. In particular, secondary weakness tags can be expanded into additional leaves and defense categories. This preserves the repair-oriented primary attribution while still representing layered defense opportunities.

Table 2: Main fields in a defense opportunity record.

Field	Meaning
playbook, stage, action	Identifier of the attack action in the playbook
attack_id	ATT&CK technique or sub-technique identifier
primary_leaf	First-failed control-point leaf used for primary attribution
primary_domain	Top-level control-point domain of the primary leaf
canonical_phase	Standard attack phase used for phase-aware scoring
primary_defense	Product categories expected to directly address the weakness
compensating_defense	Product categories that may provide secondary detection or mitigation
confidence	Confidence of the primary attribution
stage_logic	Aggregation semantics used later at the stage level

The opportunity map also records the aggregation semantics used for each playbook stage. These semantics specify how the action-level hit values within a stage should be combined during scoring. For example, a stage may represent alternative attack paths, sequentially required actions, or a loose cluster of related actions. The detailed aggregation rules are defined in the scoring model.

Table 2 summarizes the main fields used in the defense opportunity map.

Through these steps, benchmark construction turns attributed attack actions into structured defense opportunities. The resulting records connect four pieces of information required for scoring: what the attacker did, which victim-side control point first failed, which defense categories were expected to cover the action, and how the action should contribute to stage-level aggregation.

2.4 Defense Result Alignment

After the benchmark records are constructed, the evaluated defense system provides action-level observation results. Each observation describes how the defense system responded to a specific attack action in the playbook. The purpose of defense result alignment is to match these observations with the corresponding records in the defense opportunity map and convert them into normalized hit values for scoring.

We represent the defense observation trace as a set of action-level records. Each record contains the playbook identifier, the stage number, the action field used in the benchmark, and the observed result. Each defense observation is matched to the action record in the defense opportunity map with the same playbook identifier, stage number, and action field. After alignment, the observed result is associated with the action's control-point attribution, canonical phase, expected defense categories, confidence label, and stage aggregation semantics.

Table 3: Defense observation results and hit values.

Result	Hit	Meaning
PREVENTED	1.0	The attack path is architecturally removed before the action can occur.
BLOCKED	0.9	The action is attempted but synchronously interrupted by the defense system.
DETECTED	0.5	The action is observed or alerted, but not stopped.
MISSED	0.0	The action is neither prevented, blocked, nor detected.

Actions without a matched defense observation are treated as missed actions. Under this rule, an action receives a positive hit value only when the evaluated defense system reports that it was prevented, blocked, or detected. Otherwise, the action is treated as MISSED. It also keeps the benchmark comparable across different defense systems: every system is evaluated against the same set of benchmark actions, and missing observations are interpreted as lack of demonstrated coverage.

Each matched observation is then converted into one of four hit levels. PREVENTED denotes architectural prevention, where the attack path is removed before the action can be attempted. BLOCKED denotes real-time interruption, where the action is attempted but stopped by the defense system. DETECTED denotes successful observation or alerting without stopping the action itself. MISSED denotes the absence of prevention, blocking, or detection. These levels are mapped to numerical hit values used by the scoring model.

The distinction between PREVENTED and BLOCKED is important. Prevention means that the attack path is unavailable by design, such as when an egress policy makes an unauthorized destination unreachable. Blocking means that the action is observed and interrupted during execution, such as when an endpoint product terminates a malicious process. Both are stronger than detection-only outcomes, but prevention is assigned a slightly higher hit value because it does not depend on runtime recognition of the specific attack instance.

This alignment step produces a unified action-level scoring input. Each benchmark action is associated with both an expected defense opportunity and an observed defense result.

2.5 Stage-Aware Scoring Model

After defense result alignment, each benchmark action has an action-level hit value and a set of benchmark metadata, including its canonical phase, attribution confidence, and stage aggregation semantics. The scoring model uses these fields to compute defense scores at three levels: action, stage, and playbook.

The main design principle is to separate defense quality from stage importance. The stage score measures how well the defense performed within a stage, while the stage weight determines how much that stage contributes to the playbook-level score. Let a denote an attack action. Its hit value $h(a) \in [0, 1]$ is obtained from the aligned defense result described in the previous subsection.

The action weight $w(a)$ is defined as

$$w(a) = W_{\text{phase}}(\phi(a)) \cdot C_{\text{conf}}(\gamma(a)),$$

where $\phi(a)$ is the canonical phase assigned to action a, W_{phase} is the phase-weight function, $\gamma(a)$ is the attribution confidence label, and C_{conf} is the confidence factor. In our benchmark, the confidence factor reduces the contribution of actions whose attribution is less certain. For example, high-, medium-, and low-confidence actions can be assigned factors of 1.0, 0.8, and 0.5, respectively.

For a playbook stage s, let A_s be the set of actions in that stage. The stage score S_s is computed according to the aggregation semantics of the stage. We use three aggregation types: OR, AND, and Cluster.

For an OR stage, the attacker only needs one action in the stage to succeed in order to proceed. Therefore, the defense must block all actions in the stage. The stage score is defined as

$$S_s = \min_{a \in A_s} h(a).$$

This reflects the weakest-link constraint: if any action is missed, the stage is considered poorly defended.

For an AND stage, the attacker needs all actions in the stage to succeed in order to complete the stage objective. Therefore, blocking any one required action can disrupt the stage. The stage score is defined as

$$S_s = \max_{a \in A_s} h(a).$$

This reflects the strongest defensive interruption within the stage.

For a Cluster stage, the actions do not have a strong logical dependency. They are treated as a group of related behaviors rather than as strict alternatives or strict requirements. The stage score is computed as a weighted average:

$$S_s = \frac{\sum_{a \in A_s} h(a) w(a)}{\sum_{a \in A_s} w(a)}.$$

In this case, actions with higher phase importance or higher attribution confidence contribute more to the stage score.

The stage weight is computed separately from the stage score:

$$W_s = \sum_{a \in A_s} w(a).$$

This separation avoids mixing the quality of defense within a stage with the importance of that stage in the overall playbook. In particular, OR and AND stage scores are not weighted internally, because their semantics are determined by the attacker's path structure. The weights are used when aggregating stages into a playbook score.

For a playbook P with stages $S(P)$, the playbook-level score is

$$\text{Score}(P) = \frac{\sum_{s \in S(P)} W_s S_s}{\sum_{s \in S(P)} W_s}.$$

The overall defense capability score is then computed by aggregating the scores over all evaluated playbooks:

$$\text{Score}_{\text{overall}} = \frac{1}{|\mathcal{P}|} \sum_{P \in \mathcal{P}} \text{Score}(P),$$

where $\mathcal{P}$ denotes the set of playbooks in the benchmark.

This formulation keeps the final score traceable: a score loss can be traced back from the overall score to a playbook, then to a stage, and finally to the individual actions and control-point failures that caused the loss.

2.6 Multi-Dimensional Score Outputs

The scoring model does not treat the overall score as the only output. A single aggregate score can indicate whether a defense system performs well on average, but it does not explain where the score is gained or lost. To support diagnosis and comparison, the framework reports scores from multiple perspectives: overall performance, attack phase, control-point domain, defense product category, and action-level attribution.

The overall score summarizes the performance of the evaluated defense system across all playbooks. It is useful as a compact indicator, but it is not intended to be interpreted alone. The same overall score may result from different defense profiles. For example, one system may cover many attack actions but only detect them late, while another system may cover fewer actions but block them reliably within its expected scope. Therefore, the overall score is reported together with the decomposed outputs.

The phase-wise score groups results by canonical attack phase. This view answers the question: at which attack phases does the defense system lose the most score? Since the scoring model assigns higher value to earlier interception, phase-wise results help distinguish early-stage blocking from late-stage observation. This directly supports stage-aware analysis of APT defense capability.

The control-point domain score groups results by the primary victim-side control-point domain. This view answers a different question: which type of defensive control fails most often or contributes most to score loss? For example, a low score in an identity-related domain points to weaknesses in credential and trust controls, while a low score in an observability-related domain indicates insufficient detection or response. This decomposition turns an aggregate score into repair-oriented feedback.

The product category score evaluates performance within the expected coverage of each defense product category. For a product category c, let A_c denote the set of actions whose defense opportunity records include c as a primary or compensating defense category. The coverage of c is the fraction of benchmark actions that fall into this expected coverage set:

$$\text{Coverage}(c) = \frac{|A_c|}{|\mathscr{A}|},$$

where $\mathscr{A}$ is the set of benchmark actions.

The hit rate of c is computed only within its coverage set:

$$\text{HitRate}(c) = \frac{\sum_{a \in A_c} \tilde{h}_c(a)}{|A_c|}.$$

Here, $\tilde{h}_c(a)$ is the hit value credited to category c. Primary defense matches receive full hit credit, while compensating defense matches may receive discounted credit because they detect or mitigate a weakness but do not directly eliminate the failed control point.

Reporting both coverage and hit rate is important. Coverage describes how much of the benchmark a product category is expected to address. Hit rate describes how well it performs within that expected scope. A product with narrow coverage but high hit rate has a different defense profile from a product with broad coverage but low hit rate, even if their aggregate scores are similar.

Finally, the framework produces an action-level attribution report. For each score loss, the report retains the playbook, stage, action, ATT&CK technique, canonical phase, primary control-point leaf, defense categories, and observed result. This report provides the trace from aggregate score back to concrete attack actions and failed control points. As a result, the output can support both high-level comparison and detailed remediation analysis.

Table 4: Prototype validation of defensive weakness attribution.

Field	Agreement	κ	Interpretation
Scope	100.0%	1.000	Almost perfect
L1 domain	96.0%	0.953	Almost perfect
L4 leaf	86.0%	0.856	Almost perfect
Confidence	82.0%	0.679	Substantial
Mapping method	82.0%	0.687	Substantial
Secondary tag	66.0%	0.173	Slight

2.7 Prototype Method Validation

The preceding subsections define the attribution schema, benchmark construction process, result alignment procedure, scoring model, and multi-dimensional outputs. Before applying the framework to real defense-system replay experiments, we perform a set of prototype validation experiments on the benchmark and scoring infrastructure. These experiments do not evaluate a deployed defense product. Instead, they test whether the proposed methodological components are reproducible, whether the control-point–to–defense mapping is reasonable, and whether the scoring engine behaves correctly under boundary and parameter-variation cases.

First, we evaluate the reproducibility of defensive weakness attribution through a pilot inter-rater reliability study. A stratified sample of 50 ATT&CK techniques and sub-techniques was independently labeled by two annotators using the same coding guide. Agreement was measured at multiple levels, including scope decision, top-level domain, leaf-level attribution, confidence label, mapping method, and secondary weakness tag. Table 4 summarizes the results.

The result shows that the primary attribution levels are reproducible in the pilot setting. The high agreement at the L1 and L4 levels indicates that the main control-point attribution can be applied consistently. In contrast, the secondary tag has much lower agreement, so it is retained as qualitative chain-context information rather than as a primary quantitative scoring field.

Second, we validate the defense-category mapping used by the Defense–Tech Crosswalk. The external validity check compares the crosswalk categories with the dataset's native control labels. Among 1,733 comparable actions, the two sources agree on 1,297

Table 5: Prototype validation of defense-category mapping.

Validation item	Result
External agreement with native control labels	74.8% (1,297 / 1,733 actions)
Highest category agreement	DLP 96.4%, NET 93.6%, EGW 90.3%
Endpoint-category agreement	AV 73.5%, EDR 71.9%
Primary defense internal agreement	97.1% (34 / 35 leaves), $\kappa = 0.485$
Compensating defense internal agreement	45.7% (16 / 35 leaves), $\kappa = -0.094$

Table 6: Prototype scoring checks with synthetic coverage traces.

Simulation	Configuration	Score
Perfect	All actions are PREVENTED	100.00
None	All actions are MISSED	0.00
Ideal EDR	EDR primary coverage → BLOCKED	44.40
Ideal NGFW	NGFW/NET primary coverage → BLOCKED	24.93

actions, giving an agreement rate of 74.8%. We also perform an internal consistency check in which an independent annotator maps a stratified sample of 35 leaves to the 29 defense categories without seeing the existing crosswalk. Table 5 summarizes these checks.

These results suggest that the primary defense mapping is stable enough to support product-category scoring. The lower consistency for compensating defenses indicates that compensating coverage is less sharply defined than primary coverage. Therefore, compensating matches are treated as partial credit and should be interpreted more cautiously.

Third, we check the mathematical behavior of the scoring model using synthetic coverage traces. These traces are not real defense experiments; they are boundary and coverage simulations. A perfect-defense trace assigns PREVENTED to every benchmark action, and a no-defense trace assigns MISSED to every action. Two idealized product-category traces simulate systems that block all actions in their expected EDR or NGFW/NET primary coverage. Table 6 reports the resulting scores.

The perfect and none traces verify the expected upper and lower bounds of the scoring engine. The ideal EDR and ideal NGFW simulations show that a single product category cannot cover the whole benchmark even when it performs perfectly within its expected scope. This result reflects the multi-stage and multi-control nature of APT defense rather than the performance of a specific commercial product.

Finally, we test the robustness of the scoring results under reasonable parameter and aggregation variations. We vary phase weights, confidence factors, and compensating-defense discounts, and we also compare alternative aggregation choices for discovery-heavy Cluster stages and playbook-level aggregation. Table 7 summarizes the main results.

Table 7: Sensitivity and aggregation robustness checks.

Check	Result
Perfect / none under all parameter settings	100 / 0 in all configurations
EDR score range	43.41–46.67
NGFW score range	23.62–27.57
EDR/NGFW ratio range	1.57–1.94
Discovery-dominated Cluster alternative	Maximum score delta 1.07
Action-count weighted playbook aggregation	Maximum score delta 0.78

The sensitivity analysis shows that the boundary behavior of the scoring model remains stable across parameter settings, and the relative ordering between the idealized EDR and NGFW traces does not change. The aggregation robustness checks also show limited score variation under alternative but reasonable aggregation choices. These results support the use of the proposed scoring model as a stable prototype benchmark infrastructure. The later evaluation section will use real defense-system observations to test how deployed defenses perform under this methodology.

3 Evaluation

4 Discussion

5 Conclusion

References

[1] Arnau Erola, Ioannis Agrafiotis, Jason R. C. Nurse, Louise Axon, Michael Goldsmith, and Sadie Creese. 2022. A System to Calculate Cyber Value-at-Risk. *Computers & Security* 113 (2022), 102545. doi:10.1016/j.cose.2021.102545

[2] FIRST. 2019. Common Vulnerability Scoring System Version 3.1: Specification Document. https://www.first.org/cvss/v3.1/specification-document. Accessed: 2026-05-26.

[3] Marc Frigault, Lingyu Wang, Anoop Singhal, and Sushil Jajodia. 2008. Measuring Network Security Using Dynamic Bayesian Network. In *Proceedings of the 4th ACM Workshop on Quality of Protection*. ACM, 23–30. doi:10.1145/1456362.1456368

[4] Google Cloud Mandiant. 2026. M-Trends 2026: Data, Insights, and Strategies From Mandiant Frontline Investigations. https://cloud.google.com/blog/topics/threat-intelligence/m-trends-2026. Accessed: 2026-05-26.

[5] IBM Security. 2024. Cost of a Data Breach Report 2024. https://www.ibm.com/think/insights/whats-new-2024-cost-of-a-data-breach-report. Accessed: 2026-05-26.

[6] Peter E. Kaloroumakis and Michael J. Smith. 2021. Toward a Knowledge Graph of Cybersecurity Countermeasures. In *Proceedings of the 2021 Workshop on Cyber Security Experimentation and Test (CSET)*. https://d3fend.mitre.org/resources/D3FEND.pdf

[7] Pratyusa K. Manadhata and Jeannette M. Wing. 2011. An Attack Surface Metric. *IEEE Transactions on Software Engineering* 37, 3 (2011), 371–386. doi:10.1109/TSE.2010.60

[8] Peter Mell, Karen Scarfone, and Sasha Romanosky. 2006. Common Vulnerability Scoring System. *IEEE Security & Privacy* 4, 6 (2006), 85–89. doi:10.1109/MSP.2006.145

[9] MITRE. 2026. MITRE ATT&CK. https://attack.mitre.org/. Accessed: 2026-05-26.

[10] MITRE. 2026. MITRE D3FEND: A Knowledge Graph of Cybersecurity Countermeasures. https://d3fend.mitre.org/. Accessed: 2026-05-26.

[11] MITRE Engenuity. 2026. ATT&CK Evaluations: Methodology Overview. https://evals.mitre.org/methodology-overview/. Accessed: 2026-05-26.

[12] Xinming Ou, Wayne F. Boyer, and Miles A. McQueen. 2006. A Scalable Approach to Attack Graph Generation. In *Proceedings of the 13th ACM Conference on Computer and Communications Security*. ACM, 336–345. doi:10.1145/1180405.1180446

[13] Xinming Ou, Sudhakar Govindavajhala, and Andrew W. Appel. 2005. MulVAL: A Logic-based Network Security Analyzer. In *Proceedings of the 14th USENIX Security Symposium*. USENIX Association.

https://www.usenix.org/conference/14th-usenix-security-symposium/mulval-logic-based-network-security-analyzer
[14] Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. 2020. *Zero Trust Architecture.* NIST Special Publication 800-207. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-207
[15] Jerome H. Saltzer and Michael D. Schroeder. 1975. The Protection of Information in Computer Systems. *Proc. IEEE* 63, 9 (1975), 1278–1308. doi:10.1109/PROC.1975.9939
[16] Verizon Business. 2025. 2025 Data Breach Investigations Report. https://www.verizon.com/business/resources/reports/2025-dbir-data-breach-investigations-report.pdf. Accessed: 2026-05-26.
[17] Lingyu Wang, Tania Islam, Tao Long, Anoop Singhal, and Sushil Jajodia. 2008. An Attack Graph-Based Probabilistic Security Metric. In *Data and Applications Security XXII (Lecture Notes in Computer Science, Vol. 5094).* Springer, 283–296. doi:10.1007/978-3-540-70567-3_22
[18] Lingyu Wang, Anoop Singhal, and Sushil Jajodia. 2007. Measuring the Overall Security of Network Configurations Using Attack Graphs. In *Data and Applications Security XXI (Lecture Notes in Computer Science, Vol. 4602).* Springer, 98–112. doi:10.1007/978-3-540-73538-0_9
[19] Ren Zheng, Wenlian Lu, and Shouhuai Xu. 2018. Preventive and Reactive Cyber Defense Dynamics Is Globally Stable. *IEEE Transactions on Network Science and Engineering* 5, 2 (2018), 156–170. doi:10.1109/TNSE.2017.2736567

A Appendix

A.1 Related Work

This section expands the related-work discussion in the introduction. The most relevant prior work can be grouped by the question it answers: what the attacker did, whether a product responded to an attack step, what defensive techniques may counter an attack behavior, and how severe or risky a vulnerability or attack path is. These questions are closely related to APT defense evaluation, but they do not directly answer the question addressed in this paper: which victim-side control point first failed, and how should that failure affect a stage-aware defense score?

A.2 Attack Knowledge Bases and Step-Level Product Evaluation

MITRE ATT&CK provides a common language for describing adversary behavior. Its tactics, techniques, and procedures are widely used in threat intelligence, red-team planning, adversary emulation, and security product evaluation [9]. ATT&CK mainly answers the question: "What did the attacker do?"

This question is necessary, but it is not sufficient for repair-oriented defense evaluation. An ATT&CK technique label does not explain why the victim environment failed to stop the action. The same technique may succeed because of weak password policy, missing multi-factor authentication, exposed remote services, insufficient execution control, poor data protection, or weak monitoring. These causes lead to different remediation actions, even when the attacker behavior is described by the same ATT&CK technique. Our framework therefore uses ATT&CK-style action descriptions as input, but adds a separate attribution layer that maps each action to the victim-side control point that first failed.

MITRE ATT&CK Evaluations move closer to defense assessment by reporting product behavior against emulated attack steps [11]. They answer a more operational question: "Did the product observe, detect, or block this step?" This step-level view is valuable because it makes product behavior visible under a shared emulation plan.

However, a step-level matrix can tell defenders where a product responded, but it does not tell them how to attribute failures to repairable control points. It also does not directly assign different defensive value to different parts of a multi-stage playbook. Blocking an initial-access path and detecting a late exfiltration action are both important observations, but they do not have the same defensive value in an APT campaign. This paper builds on step-level observations, but converts them into stage-aware scores through control-point attribution, canonical phase weights, and stage aggregation semantics.

A.2.1 Defensive Knowledge Bases and Control Capabilities. D3FEND complements ATT&CK by organizing defensive techniques and their relations to adversary behaviors and digital artifacts [6, 10]. It answers a different question: "What defensive techniques can counter this behavior?" This makes it useful for reasoning about what defensive techniques may apply to a given attack behavior.

This capability-centered view is different from the attribution problem in this paper. D3FEND can indicate that a class of defensive techniques may counter an attack behavior, but it does not decide which victim-side control point first failed in a particular playbook action. For scoring, this distinction matters. A missed credential-dumping action, for example, may be related to endpoint monitoring, credential storage protection, privilege control, or response failure. Treating all of these as possible countermeasures does not by itself tell the benchmark which failure should be charged for the action, or which repair should be prioritized.

Our control-point attribution schema and defense-technology crosswalk use a narrower viewpoint. The attribution schema assigns each action to the first-failed control point used for scoring. The crosswalk then links that control point to the product categories expected to prevent, block, or compensate for the failure. They do not replace D3FEND; rather, they use a complementary viewpoint centered on control-point failure and repair priority.

A.2.2 Vulnerability Scoring and Attack-Path Analysis. Vulnerability scoring and attack-path analysis answer another set of questions. CVSS provides a standardized way to describe and score software vulnerability severity [2, 8]. It mainly asks: "How severe is this vulnerability?" This is important for vulnerability management, but it does not evaluate how a defense stack performs across a multi-stage APT playbook. A high vulnerability score does not indicate whether a deployed defense blocks initial access, detects execution, prevents lateral movement, or only observes the attack near the exfiltration stage.

Attack-graph methods extend the analysis from individual vulnerabilities to multi-step reachability. MulVAL and related work model how vulnerabilities, privileges, and configurations can be combined into attack paths [12, 13]. Other studies assign quantitative or probabilistic metrics to attack graphs [3, 17, 18], while cyber-risk quantification work estimates possible loss under different threat and control assumptions [1].

These methods are valuable because they make reachability, path risk, and expected loss explicit. Their unit of analysis, however, is usually a vulnerability, a configuration dependency, an attack path, or a loss distribution. In this paper, the unit of analysis is an APT action paired with an observed defense result. The action is mapped to a victim-side control-point failure, assigned a canonical phase, linked to expected defense categories, and then aggregated through stage and playbook logic. In short, attack graphs mainly

quantify attacker reachability or path risk, while our framework quantifies defense performance over a fixed set of APT playbooks.

A.2.3 Position of This Work. The distinction can be summarized by the question each line of work is designed to answer. ATT&CK asks what the attacker did. ATT&CK Evaluations ask whether a product observed, detected, or blocked a step. D3FEND asks what defensive techniques can counter an attack behavior. CVSS and attack graphs ask how severe a vulnerability is or how risky an attack path may be. This paper asks which victim-side control point first failed, and how that failure should affect a stage-aware defense score.

This positioning addresses the three limitations discussed in the introduction. First, it reduces black-box scoring by making score loss traceable to phases, actions, product categories, and control-point domains. Second, it addresses undifferentiated stage value by applying canonical phase weights and stage aggregation semantics, so that early blocking and late detection are not treated as equivalent. Third, it addresses missing attribution by linking missed or weakly handled actions to repairable victim-side control-point failures.

Received 7 October 2025; revised 24 December 2025; accepted 13 January 2026